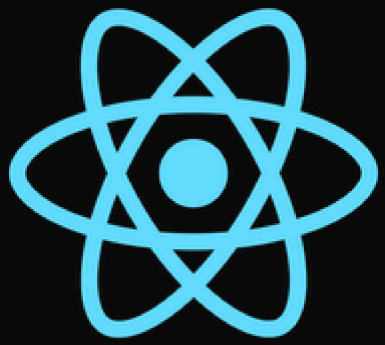
REACT



PART 1: BASICS

WHAT IS REACT? 🤔

- a JS library to create web-based UIs
- used to build Single-Page Web application

HISTORY 👩‍🎓

- created by Jordan Walke, a software engineer at Facebook, in 2011.
- used in Facebook Newsfeed and Instagram
- widely spread
- big community
- current version (2025): 19.0.0

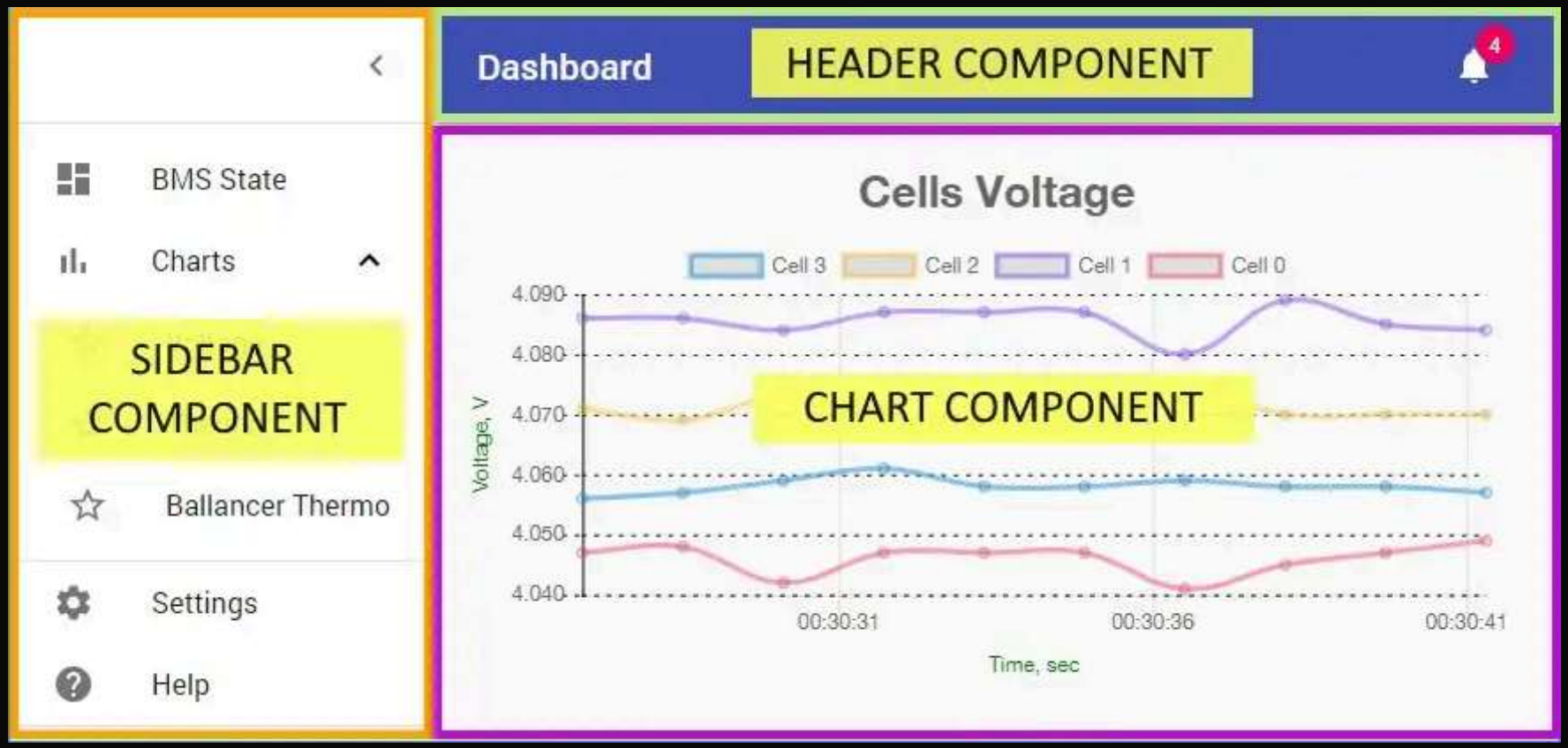
CORE CONCEPTS 🎯

- Virtual DOM (Internal optimization in React 19)
- JSX (Javascript XML)

CORE CONCEPTS 🎯

COMPONENTS:

separation of concerns for parts of an application

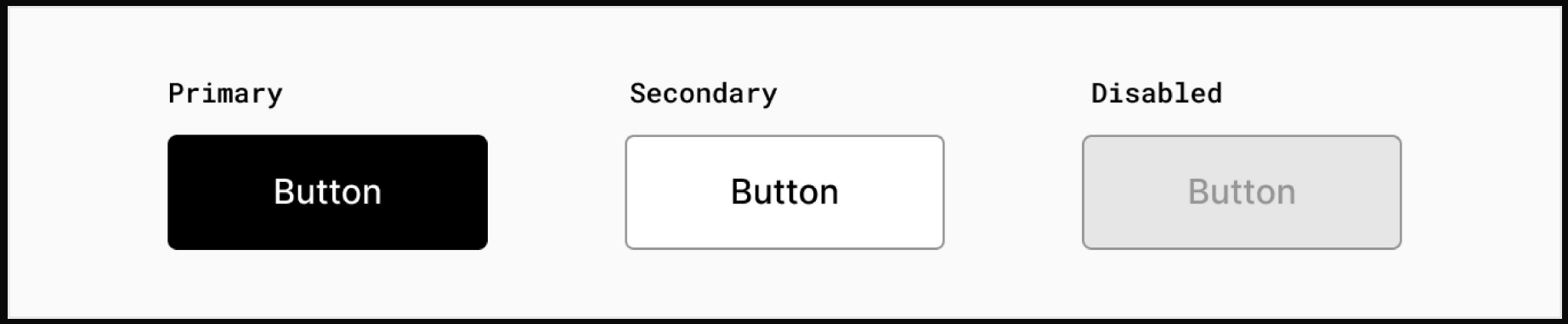


CORE CONCEPTS 🎯

PROPS:

used to pass information to React components

(❗ read-only ❗)



CORE CONCEPTS 🎯

STATE:

used to pass information & functions that can change over time (❗ write & read ❗)

React State	Props
React State is mutable and its value can be changed as per requirement.	Props are immutable, that is their content cannot be changed once assigned.
React State	**Props**
Set by parent component.	Set by event handlers that is they are completely managed by component itself.
React State	**Props**
State is local to a component and cannot be used in other components.	Props allows child components to read values from parent components.

www.educba.com

GETTING STARTED... 🏃‍♀️

CREATE A NEW PROJECT WITH VITE

```
npm create vite@latest my-react-app -- --template react
```

(Alternatively use the IDE Wizard and select Vite)

FILES & FOLDERS 🔍

- index.html (in the root folder)
- src/main.jsx (or index.js)
- src/App.jsx
- src/App.css

RUNNING THE APP

```
npm install
npm run dev
```

COMPONENTS

In React, everything is a component - a reusable piece of UI.

CLASS COMPONENTS

Traditional way of creating components using ES6 classes.

```
class MyClassComponent extends React.Component {
    render() {
        return (
<div>
    Class Component
</div>
        );
    }
}
```

In class components, the `render()` method returns the component's UI.

FUNCTIONAL COMPONENTS

Simplest form of React components, using JavaScript functions.

```
const MyComponent = () => {
    return (
        <div>
Functional Component
        </div>
    );
};
```

PROPS

Props (short for properties) are inputs passed into components.

```
class ParentComponent extends React.Component {
    render() {
        return <ChildComponent name="Lars" age= {40} />;
    }
}

class ChildComponent extends React.Component {
    render() {
        return (
<div>
    <p>Name: {this.props.name}</p>
    <p>Age: {this.props.age}</p>
</div>
        );
    }
```

STATE & HOOKS

STATE 🏚️

- State holds information about the component, and hooks allow using state in functional components.
- 👮‍♀️ **NEVER** modify states manually!
- use `setState()` or the `useState()` hook

HOOKS 🎣

- Hooks have been introduced in React 16.8.
- a useful package of functions (all of them start with `use`)
- `useState()` will be used now!
- since React 19: `useActionState` and `useOptimistic` for forms!
- enables you to make heavy use functional components 💪
- class components are not needed in many cases 🎉
- hooks use **array destructuring**

DESTRUCTURING ARRAY SYNTAX

```
const [count, setCount] = useState(0)
```

- `count` is the read value
- `setCount` is the write callback
- can be used for props and state

```
import React, { useState } from 'react';

const MyComponent = () => {
    const [count, setCount] = useState(0);

    return (
    <div>
        <p>Count: {count}</p>
        <button onClick={() => setCount(count + 1)}>
Increment
        </button>
    </div>
    );
};
```

React in 100 Seconds

USEFUL RESOURCES

- https://react.dev/learn
- https://owlypixel.com/learning-react-main-concepts/